COMP 018: Database Administration — Activity 5 Progress Report

Database Maintenance, Performance Monitoring, and Query Analysis

Project Identification & Metadata

Course	COMP 018 Database Administration
Evaluating Group	Group 4: Olivarez, Abangan, Pocsidio, Infantado, and Bartolata
Evaluating Group Repository	https://github.com/Nycri/brewpos
Target Project Monitored	CampusSafe: Campus Hazard & Safety Reporting System
Target Project Group	Group 7 (Alba, Arreza, Orpesa, Robiso, Santos)
Target Project Repository	https://github.com/jeffersonorpesa2-ops/grp07-campussafe
Primary Database	grp07_campussafedb (PostgreSQL 18.6 on x86_64 Windows, compiled by MSVC)
Restored Test Database	grp07_campussafedb_restore_test

ACTIVITY 5: Database Maintenance and Monitoring

2.1 Database Health & Overview Inspection
Database health inspection was conducted on PostgreSQL 18.6 running on a 64-bit Windows environment. Analysis confirmed that the database occupies a total disk footprint of approximately 8.9 megabytes across nine standard tables and nine sequences within the public schema. Table storage analysis showed that the hazard report table represents the largest relation at 112 kilobytes, followed by assignment records at 72 kilobytes, user profiles at 64 kilobytes, and attachment metadata at 48 kilobytes. Additional supporting tables, including hazard categories, report status histories, user roles, maintenance actions, and campus locations, range between 32 kilobytes and 48 kilobytes each.

2.2 Connection, Session & Activity Monitoring
Database performance metrics indicated six active background connections with 1,210 committed transactions and 9 rolled back transactions. Zero lock conflicts and zero deadlocks were recorded across operational sessions, yielding a transaction rollback ratio below 0.74 percent and confirming stable transaction processing. Inspection of system catalog roles confirmed that login permissions and group memberships accurately reflect the defined access control architecture.

2.3 Table Statistics & Bloat Inspection

Table usage statistics revealed that six dead tuples accumulated within the user profile table following seed initialization script executions. The hazard report and attachment tables accumulated one dead tuple each during operational testing, while remaining tables maintained zero dead tuples. The presence of obsolete row versions across active tables confirmed the necessity of performing routine tuple cleanup and statistics updating operations.

2.4 Log Review & Issue Identification

Review of active database server logs verified that unauthorized access attempts performed during role verification testing were successfully blocked and logged by the engine. Logged security events included denied insertion attempts by reporting roles, blocked update and deletion requests on hazard reports, rejected database creation commands, and blocked cross-database connection attempts. Engine operational logs recorded normal checkpoint completions without system crashes or storage errors.

2.5 Routine Maintenance Operations

Routine maintenance was conducted by executing system statistics updating and tuple vacuuming operations across target tables. An initial permission test confirmed that non-owner application runtime accounts are denied execution of maintenance commands, proving that maintenance duties require elevated administrative credentials.

Full database statistics updates executed cleanly under administrative credentials. Standard vacuuming and statistics gathering on the hazard report table scanned the underlying relation, reallocating obsolete space and updating frozen transaction IDs without table lock bottlenecks. Vacuum processing on the user profile table successfully removed six dead tuples, restoring full page efficiency.

Full table rewriting vacuum operations were deliberately omitted because such commands require exclusive table locks that block concurrent application read and write operations. Given that all tables reside within single disk pages, standard non-blocking vacuuming reclaimed dead tuples effectively without disrupting application availability.

2.6 Query Performance Analysis

Performance analysis was conducted on an administrative query filtering hazard reports by submission status and ordering results by timestamp. The query executed in 0.062 milliseconds with complete buffer cache hit efficiency. Although an index exists on the status column, the query optimizer selected a sequential scan over an index lookup. Because the underlying data resides within a single memory buffer page, scanning the block directly avoided the overhead of index page processing and pointer lookups.

Analysis of a second relational query joining hazard reports with category and location tables yielded an execution time of 0.069 milliseconds with zero physical disk reads. The query engine utilized in-memory hash joins across category and location lookup tables, processing joined data streams within dedicated memory structures without disk spills.

2.7 Prioritized Recommendations & Technical Justification

The existing index structure is appropriate for current operational requirements, as active queries execute under 0.1 milliseconds with full buffer cache hits. Immediate index modifications are unnecessary and would add write overhead during record insertions. Should incident record volume grow significantly in production, establishing a composite index covering report status and submission timestamps will maintain query efficiency. Additionally, establishing a scheduled background maintenance job will ensure regular dead tuple removal during periods of low application activity.

CLASS GOOGLE SHEETS TRACKER ENTRY (Activity 5)

Column Header	Value
Project	Group 7 - CampusSafe: Campus Hazard & Safety Reporting System
Database inspected	grp07_campussafedb (PostgreSQL 18.6)
Account and role used	postgres (DBA Maintenance), campussafe_app (Runtime), campussafe_report (Audit)
Health check result	Healthy (0 conflicts, 0 deadlocks, 8.9 MB footprint, 100% buffer cache hits)
Maintenance performed	ANALYZE; VACUUM (VERBOSE, ANALYZE) on hazard_report, report_status_history, app_user via postgres
Query analyzed	Admin status filter with sort (Seq scan, 0.062 ms, 4 buffer hits) & Multi-table join (Hash joins, 0.069 ms)
Recommendation priority	No action required (Current queries execute in < 0.07 ms with 100% buffer cache hits)
Evidence folder link	https://github.com/Nycri/brewpos/tree/main/activity-5/Evidence-act5
Feedback sent to owners	(05/11/2026)
PDF progress document	Activity5_Progress_Report_Group7.pdf
Last updated	2026-11-05

AI USAGE DECLARATION

Google Gemini 3.8 Flash was utilized during project evaluation to inspect database schema definitions, evaluate relational hierarchy structures, formulate role verification test scenarios, analyze database maintenance operations and query performance execution plans, verify backup creation and restoration procedures, and structure technical report documentation. All database statements, configuration checks, and verification procedures were directly executed and validated against PostgreSQL 18.6 within the local database environment.